

FRONT PAGE

Nombre Alumno / DNI	Óscar Ortega Rueda / 54291152C
Título del Programa	1º PHE CYBER / APPLIED COMPUTING / DATA SCIENCE
N.º Unidad y Título	UNIT 15 - Artificial Intelligence & Intelligent Systems
Año académico	2025-2026
Profesor de la unidad	Jose Francisco
Título del Assignment	UNIT 15 ARTIFICIAL INTELLIGENCE & INTELLIGENT SYSTEMS
Día de emisión	23/10/2025
Día de entrega	29/01/2026
Declaración del estudiante	Certifico que la presentación del assignment es completamente mi propio trabajo y entiendo completamente las consecuencias del plagio. Entiendo que hacer una declaración falsa es una forma de mala práctica. Fecha: 28/01/2026 Firma del alumno: 

Plagio

El plagio es una forma particular de hacer trampa. El plagio debe evitarse a toda costa y los alumnos que infrinjan las reglas, aunque sea inocentemente, pueden ser sancionados. Es su responsabilidad asegurarse de comprender las prácticas de referencia correctas. Como alumno de nivel universitario, se espera que utilice las referencias adecuadas en todo momento y mantenga notas cuidadosamente detalladas de todas sus fuentes de materiales para el material que ha utilizado en su trabajo, incluido cualquier material descargado de Internet. Consulte al profesor de la unidad correspondiente o al tutor del curso si necesita más consejos.

**Para el siguiente ejercicio dispondremos de varios archivos en formato CSV que contienen información sobre embalses, fechas y cuantías de agua. Debemos unir correctamente la información de cada uno de los archivos en un dataframe. No se pueden eliminar valores que contengan información esencial. Posteriormente deberemos unir la cuantía de agua de cada embalse en una columna, de esta forma obtendremos un índice, el cual debe ser el nombre del embalse y cada dato de la cuantía de agua debe quedar en su misma posición vertical. Los índices de las filas indicarán el año y el mes para cada valor, ordenando así los valores de agua de cada embalse por fecha.

Fase 1: Concatenación y generación del dataframe

**El dataframe debe quedar de la siguiente forma

	anio	mes	ElAtazar	ElVado	ElVillar	LaAcena	LaJorosa	LaPinilla	LosMorales	Navacerrada	Navalmedio	Pedrezuela	PuentesViejas	Riosequillo	Santillana	Valmayor	TOTAL	TOTAL_float
0	1998	Enero	326,784	45,882	22,741	23,891	6,572	30,331	2,373	10,062	0,182	38,413	41,769	45,782	76,953	118,99	790,725	790.725
1	1998	Febrero	327,59	46,043	21,55	23,816	6,462	30,254	2,37	10,114	0,617	38,104	41,442	46,64	78,44	119,578	793,020	793.020
2	1998	Marzo	336,267	39,928	21,67	23,245	6,848	30,02	2,363	9,966	0,5	33,449	40,911	45,965	78,067	119,063	788,262	788.262
3	1998	Abril	353,267	42,443	21,644	23,35	6,747	31,674	2,363	10,326	0,639	27,848	44,173	46,209	82,716	119,578	812,977	812.977
4	1998	Mayo	391,448	49,165	21,925	23,678	7,041	35,518	2,338	10,989	0,625	23,009	50,395	43,954	85,453	120,833	866,371	866.371
Distribución por fechas			...	Máximo	...	...	...	...	...	...	...	...	Valor mayor de cada embalse			Total de cada embalse		
283	2021	Agosto	Mínimo	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	0,000	NaN
284	2021	Septiembre	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	0,000	NaN
285	2021	Octubre	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	0,000	NaN
286	2021	Noviembre	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	0,000	NaN
287	2021	Diciembre	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	0,000	NaN

**A continuación se analiza cada paso para realizar la primera fase del ejercicio y, por último se analiza la estructura del código

**El programa que utilizaremos será Google Colab debido a su facilidad de uso y su versatilidad para realizar diferentes pruebas.

Preparación de documentos

**Importación de librerías necesarias y motaje de Drive en el sistema. Posteriormente, mediante los comandos ls, cd y cd.. navegaremos hasta la carpeta donde se encuentran los archivos CSV.

```
[1] 16 s
from google.colab import drive
import pandas as pd
import os as os
from google.colab import files

drive.mount('/content/dive')

... Mounted at /content/dive

[2] 0 s
cd /content/dive/MyDrive/csv

/content/dive/MyDrive/csv
```

**Visualización de archivos CSV

```
ls

AguaEmbalsada_RioCofio_LaAcena.csv
AguaEmbalsada_RioGuadalix_Pedrezuela.csv
AguaEmbalsada_RioGuadarrama-Aulencia_LaJorosa.csv
AguaEmbalsada_RioGuadarrama-Aulencia_Navalmedio.csv
AguaEmbalsada_RioGuadarrama-Aulencia_Valmayor.csv
AguaEmbalsada_RioJarama_ElVado.csv
AguaEmbalsada_RioLosMorales_LosMorales.csv
AguaEmbalsada_RioLozoya_ElAtazar.csv
AguaEmbalsada_RioLozoya_ElVillar.csv
AguaEmbalsada_RioLozoya_LaPinilla.csv
AguaEmbalsada_RioLozoya_PuentesViejas.csv
AguaEmbalsada_RioLozoya_Riosequillo.csv
AguaEmbalsada_RioManzanares_Navacerrada.csv
AguaEmbalsada_RioManzanares_Santillana.csv
```

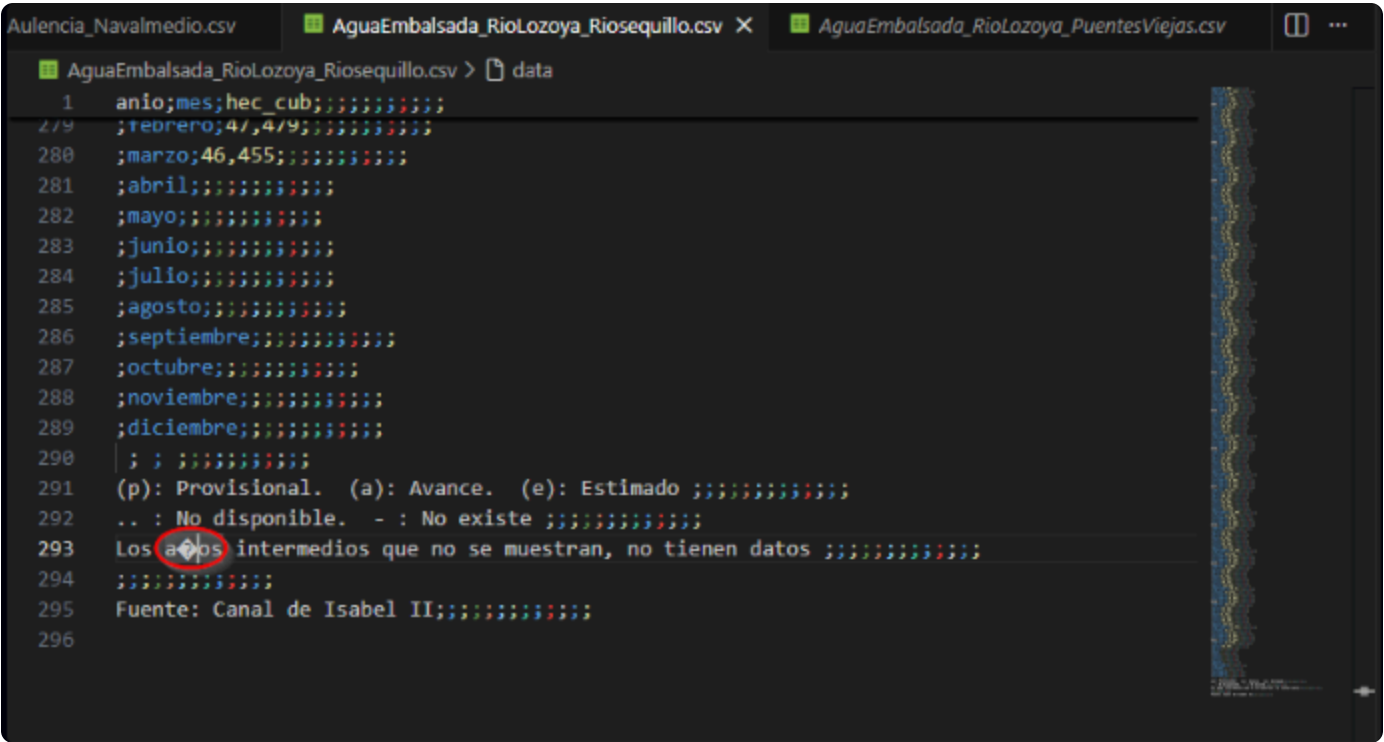
**Limpieza de caracteres inválidos para el tratado del archivo, así como los caracteres no compatibles con UTF-8

```
import os

ruta = "/content/dive/MyDrive/csv"

for archivo in os.listdir(ruta):
    if archivo.lower().endswith(".csv"):
        path_archivo = os.path.join(ruta, archivo)
        with open(path_archivo, "rb") as f:
            for i, line in enumerate(f, 1):
                try:
                    line.decode("utf-8")
                except UnicodeDecodeError:
                    print(f"[ERROR] {archivo}, línea {i} contiene caracteres no UTF-8")
```

****Se identificaron y modificaron los siguientes caracteres inválidos**



Análisis del código funcional

****El siguiente código define los valores generales para cada índice vertical y horizontal, los ordena y mantiene un estándar para el tratado de datos y su compatibilidad, por ejemplo, para cada índice todo en minúsculas y sin espacios, una vez concatenado, se pueden modificar, sirve para marcar estándares para la concatenación.**

****Una vez se definen los estándares y se manejan los errores se procede a concatenar. Es importante añadir que para cada modificación de orden se debe resetear el índice para evitar fallos, el índice numérico muestra al sistema que valor hay en cada fila y columna en específico, si se modifica, crearemos que estamos modificando un valor que no es correcto, por ejemplo, puede que queramos cambiar las columnas año por mes y estemos cambiando un valor de hec_cub por un nombre de un embalse.**

**En el siguiente código se explica linea por linea el funcionamiento de este, posteriormente se analiza la versatilidad y características de forma general

```
# Importación de librerías necesarias

import os # Manejo de carpetas en bucles etc.

import pandas as pd # Manejo de archivos CSV y unión de cada uno en un Dataframe

from google.colab import files # Opción de descarga del archivo final


# Función que unirá los archivos CSV en un Dataframe una vez limpiados


def generar_dataframe_embalses_completos():

    ruta = "/content/dive/MyDrive/csv" # Definición de ruta archivos CSV

    ruta_descarga = "/content/dive/MyDrive/dataset_embalses_final.csv"

    # Ruta de descarga del CSV final
```

```
# Definición de los meses válidos una vez revisados, todos coinciden

meses_validos = [

    "enero", "febrero", "marzo", "abril", "mayo", "junio",

    "julio", "agosto", "septiembre", "octubre", "noviembre", "diciembre"

]


# Definición de lista donde se guardará cada embalse

df_list = []


# Bucle que recorre cada archivo y lo guarda en la lista anterior

for archivo in os.listdir(ruta):

    if not archivo.lower().endswith(".csv"):

        continue

    try: # Try para limpieza de datos

        embalse = archivo.replace(".csv", "").split("_")[-1] # Eliminación de espacios y barras ba

        df = pd.read_csv(

            os.path.join(ruta, archivo),

            sep=";", # Separación de columnas mediante ;

            encoding="utf-8-sig", # Utilizar compatibilidad de caracteres UTF-8

            engine="python" # Utilizar motor de python

        )

        # Eliminación de espacios y conversión a minúsculas de cada índice de columna

        df.columns = df.columns.str.strip().str.lower()

        # Definición de como deben quedar los índices de las columnas

        columnas_de_interes = ['anio', 'mes', 'hec_cub']

        # Verifica si los datos tratados son iguales a las columnas de la linea anterior

        df = df[[c for c in columnas_de_interes if c in df.columns]]

        # Limpiar columnas (Elimina espacios, convierte todo a minúculas y a formato string)

        df['mes'] = df['mes'].astype(str).str.strip().str.lower()

        df['anio'] = pd.to_numeric(df['anio'], errors='coerce') # años faltantes con valor NaN

        df['hec_cub'] = df['hec_cub'].astype(str).str.strip() # mantener coma

        # Propagar años hacia abajo por si acaso recibimos otros csv con errores en un futuro
```

```
df['anio'] = df['anio'].ffill()

# Filtrar meses válidos de nuevo por si recibimos nuevos archivos

df = df[df['mes'].isin(meses_validos)]

# Convertir año a formato int

df['anio'] = df['anio'].astype(int)

df['embalse'] = embalse # Crear columna embalse y asignar el nombre real de cada embalse a cada
índice de columna embalses

df_list.append(df) # Añadir datos a la lista para actualizar el dataset si obtenemos nuevos
archivos

# Manejo de errores obtenidos en las operaciones con que intentó Try

except Exception as e:

    print(f"[ERROR] {archivo}: {e}") # Explica el error y su tipo

if not df_list: # Si la lista esta vacía porque fallaron operaciones en cada archivo, entonces devuelve
none (nada)

    print("No se encontraron datos válidos.") # Aviso de que no hay nada en la lista de embalses (las
operaciones con try fallaron)

    return None

# Concatenar todos los CSV con uso de pandas y .concat

df_unido = pd.concat(df_list, ignore_index=True) # Concatena y actualiza el índice para el nuevo
documento

# Pivotar valores para que cada valor este en la columna de embalse

df_pivot = df_unido.pivot_table(

    index=['anio', 'mes'], # Definición de valores para las filas

    columns='embalse', # Definición del índice de la columna embalse, con el nombre original de cada uno

    values='hec_cub', # Definición de valores de cada embalse

    aggfunc=lambda x: ','.join(x) # aggfunc trata valores duplicados para el mismo mes, en conjunto, se
unen los valores como único string

).reset_index() # Resetea a un nuevo índice

# Ordenar meses

df_pivot['mes'] = df_pivot['mes'].str.capitalize() # Convierte la primera letra de cada mes en
mayúsculas y a string

df_pivot['mes'] = pd.Categorical( # Obliga a manetener un orden de los meses, primero Enero, después
Febrero, etc.
```

```

        df_pivot['mes'],

        categories=[m.capitalize() for m in meses_validos], # realizar las lineas anteriores en todos los
datos

        ordered=True # Define que ya esta ordenado y que el orden es como se definió, por si hay un error y
se revisa el orden

    )

    df_pivot = df_pivot.sort_values(['anio', 'mes']).reset_index(drop=True) # Ordenar índices de filas
primero por año y después por mes y reseteo del index

# Crear columna TOTAL sumando numéricamente para conseguir el total del mínimo y el máximo

columnas_embalses = df_pivot.columns.difference(['anio', 'mes'])

# Convertir a float temporal para sumar

df_pivot['TOTAL'] = df_pivot[columnas_embalses].replace(',', '.', regex=True).astype(float).sum(axis=1)

# Convertir TOTAL de nuevo a string con coma

df_pivot['TOTAL'] = df_pivot['TOTAL'].map(lambda x: f"{x:.3f}".replace('.', ','))

# Convierte la columna llamada anio a año
df_pivot = df_pivot.rename(columns={'anio': 'año'})

# Guardar CSV

df_pivot.to_csv(ruta_descarga, sep=";", index=False, encoding="utf-8-sig")

print(f"Dataset generado: {ruta_descarga}")

return df_pivot # Devolver el resultado de toda la función

# Renombrar variable final y, aparte, descargar en CSV

df_final = generar_dataframe_embalses_completos()

files.download("/content/dive/MyDrive/dataset_embalses_final.csv")

```

****Si analizamos el código por completo y lo entendemos podemos extraer la siguiente conclusión.**

1. El código esta preparado para recibir nuevos archivos con el mismo número de columnas, nombres de índices de columnas y filas y valores correctos, como por ejemplo valores numéricos de `hec_cub`.
2. Es capaz de ordenar por embalse y extraer los valores máximos de cada uno y el valor total de agua.
3. El código esta preparado para manejar errores y explicarlos, marcar estándares de valores de índices para su concatenación, por ejemplo, si un archivo contiene el valor de ANIO como columna, el programa lo convierte a anio.

****Podemos observar el siguiente dataframe, convertido a archivo CSV**

```
1 año;mes;ElAtazar;ElVado;ElVillar;LaAcena;LaJorosa;LaPinilla;LosMorales;Navacerrada;Navalmedio;Pedrezuela;PuentesViejas;Riosequillo;Santillana;Valmayor;TOTAL
2 1998;Enero;326,784;45,882;22,741;23,891;6,572;30,331;2,373;10,062;0,182;38,413;41,769;45,782;76,953;118,99;790,725
3 1998;Febrero;327,59;46,043;21,55;23,816;6,462;30,254;2,37;10,114;0,617;38,104;41,442;46,64;78,44;119,578;793,020
4 1998;Marzo;336,267;39,928;21,67;23,245;6,848;30,02;2,363;9,966;0,5;33,449;40,911;45,965;78,067;119,063;788,262
5 1998;Abril;353,267;42,443;21,644;23,35;6,747;31,674;2,363;10,326;0,639;27,848;44,173;46,209;82,716;119,578;812,977
6 1998;Mayo;391,448;49,165;21,925;23,678;7,041;35,518;2,338;10,989;0,625;23,009;50,395;43,954;85,453;120,833;866,371
7 1998;Junio;412,483;41,751;19,654;21,947;6,718;29,596;2,326;9,647;0,559;21,13;43,927;41,359;78,627;109,389;839,113
8 1998;Julio;405,935;24,442;12,448;18,924;6,072;28,418;2,212;7,982;0,643;14,083;43,706;43,338;77,602;95,295;781,100
9 1998;Agosto;388,735;13,855;11,913;15,802;5,53;26,073;2,047;6,183;0,584;13,737;43,051;37,611;76,4;86,685;728,206
10 1998;Septiembre;366,046;8,98;10,276;13,085;5,881;22,086;0,984;5,051;0,336;12,905;42,358;34,818;76,216;81,99;681,012
11 1998;Octubre;336,358;13,34;13,147;11,208;6,006;21,86;1,855;4,348;0,191;10,461;34,594;35,689;76,307;78,129;643,493
12 1998;Noviembre;306,506;12,72;11,903;9,802;5,957;22,839;1,849;3,635;0,257;9,17;32,867;38,217;77,046;74,97;607,738
13 1998;Diciembre;277,656;16,277;14,065;8,315;6,095;23,005;1,801;3,277;0,37;7,679;32,255;40,743;79,096;71,615;582,249
14 1999;Enero;254,298;21,14;14,308;9,5;5,794;25,399;1,813;3,593;0,546;6,745;34,359;43,806;78,721;67,243;567,265
15 1999;Febrero;232,187;19,967;13,669;11,823;5,192;29,289;1,771;3,854;0,576;7,182;37,832;45,116;79,472;64,38;552,310
16 1999;Marzo;235,053;22,558;19,629;12,75;5,93;30,02;1,725;5,266;0,203;6,22;41,049;45,904;79,756;59,91;565,973
17 1999;Abril;244,001;24,412;19,654;11,733;6,624;30,331;1,651;5,439;0,543;4,874;41,049;45,995;78,347;59,351;574,004
18 1999;Mayo;262,633;28,557;19,566;12,366;5,848;30,098;1,609;7,104;0,333;5,96;41,141;46,179;78,354;69,764;609,512
19 1999;Junio;257,007;19,771;19,202;9,996;6,178;28,455;1,517;6,922;0,494;5,735;40,819;43,836;72,32;77,447;589,699
20 1999;Julio;236,235;14,06;17,371;5,747;6,274;26,216;1,295;6,215;0,516;5,614;40,386;45,206;63,954;77,238;546,327
21 1999;Agosto;216,535;9,749;15,576;3,866;5,457;25,611;1,084;4,497;0,471;5,43;39,889;44,577;55,169;73,102;501,013
22 1999;Septiembre;196,024;7,881;15,212;3,627;4,535;25,505;0,905;3,268;0,47;5,395;39,443;44,547;48,825;76,201;471,838
23 1999;Octubre;187,777;17,827;14,653;3,807;4,292;26,395;0,928;3,164;0,365;8,807;40,454;39,696;49,349;82,758;480,272
24 1999;Noviembre;180,915;23,233;15,103;4,668;3,885;23,708;0,869;3,071;0,241;9,739;40,454;40,079;45,627;86,399;477,991
25 1999;Diciembre;188,088;37,434;17,786;6,1;3,893;23,005;0,86;3,51;0,409;9,311;39,244;39,914;44,105;89,521;503,180
26 2000;Enero;197,154;33,373;18,633;7,496;3,669;25,223;0,856;3,432;0,497;11,061;41,257;40,052;42,682;91,593;516,978
27 2000;Febrero;203,813;27,601;18,462;7,807;4,006;27,895;0,849;3,446;0,368;13,737;43,172;41,5;40,247;92,733;525,636
28 2000;Marzo;209,496;15,738;18,206;7,392;3,582;28,568;0,813;3,51;0,239;14,083;43,1;43,017;35,839;91,652;515,235
29 2000;Abril;276,197;49,045;22,618;12,366;5,364;32,968;0,997;7,712;0,347;26,293;47,995;44,19;51,95;101,887;679,929
30 2000;Mayo;330,917;48,306;18,768;16,081;6,274;32,438;1,067;9,673;0,353;30,875;46,162;41,528;58,011;100,592;741,045
31 2000;Junio;336,905;36,247;18,122;16,345;4,665;31,554;0,981;8,595;0,442;22,549;42,931;42,411;49,716;86,971;698,434
32 2000;Julio;314,487;36,917;17,288;13,829;4,494;29,06;0,843;6,86;0,485;16,02;38,804;42,096;39,542;75,378;636,103
33 2000;Agosto;286,672;36,956;12,399;11,786;3,889;25,859;0,716;4,937;0,483;15,644;39,532;41,5;32,612;65,339;578,324
34 2000;Septiembre;258,407;37,036;12,399;10,891;3,145;22,872;0,52;3,156;0,465;15,399;36,945;39,75;26,441;56,242;523,668
35 2000;Octubre;241,596;27,261;16,707;6,508;6,296;20,781;0,438;2,063;0,418;15,278;35,228;36,346;19,732;51,139;479,791
36 2000;Noviembre;243,023;30,117;17,359;6,637;5,794;27,821;0,421;3,067;0,612;15,118;39,133;40,632;22,794;57,154;509,682
37 2000;Diciembre;291,578;41,528;22,672;12,814;5,778;30,527;0,716;6,668;0,227;33,339;41,003;41,331;49,569;85,888;663,638
38 2001;Enero;329,746;48,188;20,486;23,625;6,078;30,176;1,713;9,161;0,186;35,033;41,234;41,897;80,418;114,292;782,233
39 2001;Febrero;338,824;44,14;19,667;23,763;5,65;30,37;2,375;8,803;0,352;35,033;42,382;40,994;80,134;113,086;785,573
40 2001;Marzo;361,541;47,388;18,793;23,795;6,089;32,601;2,385;9,153;0,304;36,841;45,094;45,934;84,272;112,662;826,852
41 2001;Abril;385,338;47,857;19,73;23,742;5,37;32,682;2,371;9,013;0,359;34,81;45,145;43,895;82,426;104,253;836,991
42 2001;Mayo;408,006;37,695;17,324;23,721;4,97;32,968;2,361;8,181;0,488;29,629;40,159;40,743;80,893;94,313;821,451
43 2001;Junio;417,312;24,054;13,238;23,488;2,961;32,276;2,266;6,131;0,358;18,248;27,943;39,532;74,299;82,154;764,260
44 2001;Julio;410,084;9,274;11,836;17,716;5,736;29,865;2,09;3,908;0,304;11,871;26,665;34,134;66,287;70,247;700,017
45 2001;Agosto;380,378;8,662;12,311;15,242;5,187;26,684;1,889;2,45;0,225;11,662;27,116;28,661;59,379;59,608;639,454
46 2001;Septiembre;349,924;8,262;14,255;14,979;3,562;23,81;1,75;1,906;0,215;11,53;26,68;25,069;53,24;50,105;585,287
47 2001;Octubre;321,175;9,933;18,182;15,005;3,024;18,985;1,755;2,112;0,346;12,132;28,382;26,598;52,555;46,8;556,984
```

**Al final del documento se encuentran valores nulos, los cuales no han sido eliminados debido a la falta de información de porqué estos valores están nulos. ¿Se rellenaran?

**No obstante habrá que tratarlo en la siguiente fase del ejercicio

```
281 2021;Abril;nan;nan;nan;nan;nan;nan;;nan;nan;nan;nan;nan;nan;nan;0,000
282 2021;Mayo;nan;nan;nan;nan;nan;nan;;nan;nan;nan;nan;nan;nan;nan;0,000
283 2021;Junio;nan;nan;nan;nan;nan;nan;;nan;nan;nan;nan;nan;nan;nan;0,000
284 2021;Julio;nan;nan;nan;nan;nan;nan;;nan;nan;nan;nan;nan;nan;nan;0,000
285 2021;Agosto;nan;nan;nan;nan;nan;nan;;nan;nan;nan;nan;nan;nan;nan;0,000
286 2021;Septiembre;nan;nan;nan;nan;nan;nan;;nan;nan;nan;nan;nan;nan;nan;0,000
287 2021;Octubre;nan;nan;nan;nan;nan;nan;;nan;nan;nan;nan;nan;nan;nan;0,000
288 2021;Noviembre;nan;nan;nan;nan;nan;nan;;nan;nan;nan;nan;nan;nan;nan;0,000
289 2021;Diciembre;nan;nan;nan;nan;nan;nan;;nan;nan;nan;nan;nan;nan;nan;0,000
290
```

Fase 2: Arlgoritmo de previsión, estadísticas, preguntas y conclusiones

**Para la fase del ejercicio debemos crear un algoritmo de previsión (Machine Learning) usando prophet. Adicionalmente se crearán varias estadísticas usando matplotlib y, por último se responderán a las siguientes preguntas:

- 1.- ¿Que embalse tiene más diferencia en su max y min de agua?
- 2.- ¿Cuándo se produce sequía?, si esta se llega a producir
- 3.- ¿Podemos asegurar una respuesta segura a la pregunta anterior con los datos que tenemos?

Código para la generación de la previsión usando Prophet

```
# 1. IMPORTACIONES

import pandas as pd

import matplotlib.pyplot as plt

from prophet import Prophet

# 2. CARGA DEL DATAFRAME
```

```
df = df_final.copy()

# 3. PREPARACIÓN DE DATOS PARA PROPHET

# Diccionario para convertir meses a número

meses_dict = {

    "Enero": 1, "Febrero": 2, "Marzo": 3, "Abril": 4,

    "Mayo": 5, "Junio": 6, "Julio": 7, "Agosto": 8,

    "Septiembre": 9, "Octubre": 10, "Noviembre": 11, "Diciembre": 12

}

# Convertir mes a número

df['mes_num'] = df['mes'].map(meses_dict)

# Crear columna de fecha (ds)

df['ds'] = pd.to_datetime(

    df['año'].astype(str) + "-" +

    df['mes_num'].astype(str) + "-01"

)

# Convertir TOTAL a float

df['y'] = df['TOTAL'].str.replace(',', '.').astype(float)

# Dataset base para Prophet, lee el documento, lo ordena por columna y resetea el índice

df_prophet = df[['ds', 'y']].sort_values('ds').reset_index(drop=True)

# 4. LIMPIEZA DE DATOS Y ANOMALÍAS

# Eliminar valores nulos y ceros para mejorar la previsión

df_prophet = df_prophet.dropna()

df_prophet = df_prophet[df_prophet['y'] > 0]

# 5. CREACIÓN Y ENTRENAMIENTO DEL MODELO
```

```
model = Prophet()

model.fit(df_prophet)


# 6. GENERACIÓN DE FECHAS FUTURAS


future_dates = model.make_future_dataframe(

    periods=12,

    freq='MS' # Inicio de mes

)


# 7. PREDICCIÓN


predictions = model.predict(future_dates)


# 8. VISUALIZACIÓN DE RESULTADOS


fig, ax = plt.subplots(figsize=(14, 6))

model.plot(predictions, ax=ax)


ax.set_title("Predicción del volumen total de agua en embalses (12 meses)")

ax.set_xlabel("Fecha")

ax.set_ylabel("Volumen total (hm³)")


plt.show()


# 9. MOSTRAR SOLO LAS PREDICCIONES FUTURAS


# Obliga al algoritmo a no mezclar la predicción futura con los datos hstóricos


predicciones_futuras = predictions[

    predictions['ds'] > df_prophet['ds'].max()

][['ds', 'yhat', 'yhat_lower', 'yhat_upper']]


predicciones_futuras
```

****Este modelo de predicción es un modelo básico que predice la cuantía de agua para cada embalse en un tiempo de 12 meses (próximos).**

****Primero se define el dataframe usado para realizar la previsión, posteriormente se eliminan anomalías (Valores 0), seguidamente se define el dataset que utilizará prophet, en esta sección se ordenan las columnas y se resetean los índices. Posteriormente se crea un diccionario con los meses, usado para convertirlo a números en las líneas consiguientes, por último, se crea una columna donde se definen las fechas (formato texto, solo años y no meses ni días)**

```
# 1. Importaciones

import pandas as pd

import matplotlib.pyplot as plt

from prophet import Prophet


# 2. CARGA DEL DATAFRAME


df = df_final.copy()


# 3. PREPARACIÓN DE DATOS PARA PROPHET


# Diccionario para convertir meses a número

meses_dict = {

    "Enero": 1, "Febrero": 2, "Marzo": 3, "Abril": 4,

    "Mayo": 5, "Junio": 6, "Julio": 7, "Agosto": 8,

    "Septiembre": 9, "Octubre": 10, "Noviembre": 11, "Diciembre": 12

}


# Convertir mes a número

df['mes_num'] = df['mes'].map(meses_dict)


# Crear columna de fecha (ds)

df['ds'] = pd.to_datetime(

    df['año'].astype(str) + "-" +

    df['mes_num'].astype(str) + "-01"

)


# Convertir TOTAL a float

df['y'] = df['TOTAL'].str.replace(',', '.').astype(float)


# Dataset base para Prophet, lee el documento, lo ordena por columna y resetea el índice

df_prophet = df[['ds', 'y']].sort_values('ds').reset_index(drop=True)
```

****En la siguiente sección se crea y entrena el modelo.**

```
# 5. CREACIÓN Y ENTRENAMIENTO DEL MODELO

# Se crea el modelo

model = Prophet()

# Se entrena el modelo con los datos históricos

model.fit(df_prophet)
```

****En la siguiente sección se preparan los datos visuales de los meses y años para la tabla o estadística visual, se realiza la predicción y se definen los parámetros para la estadística como por ejemplo, su título, nombre general de columnas y de filas.**

```
# 6. GENERACIÓN DE FECHAS FUTURAS

# Se genera el dataframe de fechas futuras

future_dates = model.make_future_dataframe(

    periods=12,

    freq='MS' # Inicio de mes

)

# 7. PREDICCIÓN

# Se realiza la predicción

predictions = model.predict(future_dates)

# 8. VISUALIZACIÓN DE RESULTADOS

# Se genera la figura

fig, ax = plt.subplots(figsize=(14, 6))

# Se genera la predicción

model.plot(predictions, ax=ax)

# Se genera el título

ax.set_title("Predicción del volumen total de agua en embalses (12 meses)")

# Se genera el eje x

ax.set_xlabel("Fecha")

# Se genera el eje y

ax.set_ylabel("Volumen total (hm³)")

# Se muestra la figura

plt.show()
```

****Por último se obliga al algoritmo a no mezclar la predicción futura con los datos hstóricos y se ejecuta la gráfica predictiva**

```
# 9. MOSTRAR SOLO LAS PREDICCIONES FUTURAS

# Obliga al algoritmo a no mezclar la predicción futura con los datos hstóricos

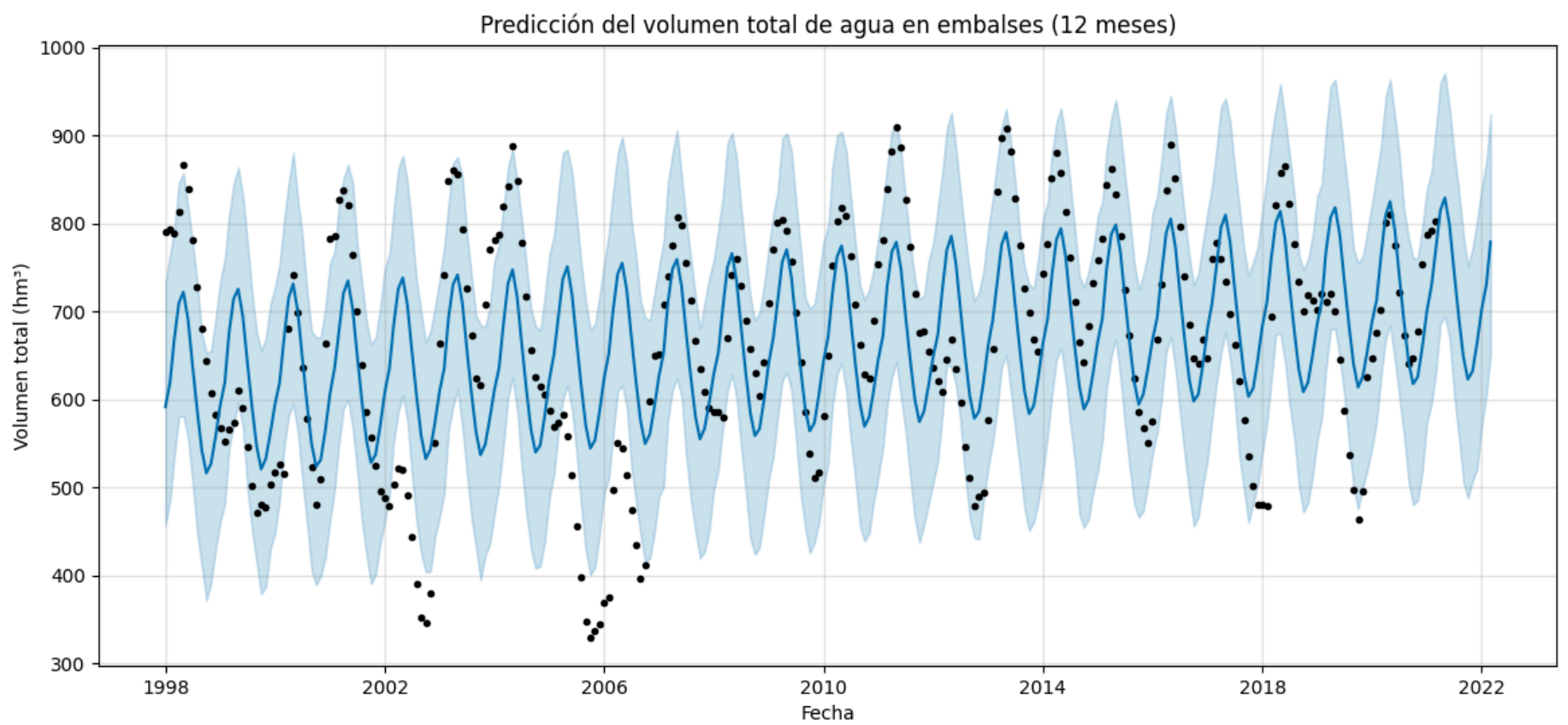
# Se genera el dataframe de predicciones futuras

predicciones_futuras = predictions[

    predictions['ds'] > df_prophet['ds'].max()

][['ds', 'yhat', 'yhat_lower', 'yhat_upper']]
```

**El resultado es el siguiente



**Puntos negros: Representan los datos hitóricos del dataframe anterior

**Línea azul: Muestra el valor de predicción real

**Banda azul transparente: Muestra los valores máximos y mínimos entre los que puede llegar a fluctuar la predicción real

**Al observar la gráfica podemos analizar que en el año 2002-2003 se produjo una sequía según los datos históricos, en el año 2006 también pasa lo mismo.

**Al buscar información en internet encontré lo siguiente:

En 2006, los embalses en España registraron niveles muy bajos debido a una severa sequía, cerrando el año hidrológico con aproximadamente el (39%) de su capacidad total. La reserva hídrica se mantuvo inusualmente baja durante gran parte del año, con embalses importantes como el de Alarcón al (5,37%) en diciembre de 2006.

En 2002, los embalses en España, particularmente en la cuenca del Júcar-Túria y la cabecera del Tajo (Entrepeñas-Buendía), experimentaron una situación crítica debido a una fuerte sequía. Las reservas descendieron a niveles muy bajos, situándose en torno al 30-45% de su capacidad, lo que provocó restricciones en el suministro y la reducción de envíos de agua.

**Podemos eliminar estos valores, no obstante, la propia gráfica está estabilizando estos valores de predicción, ya que la línea azul y su banda transparente no bajan de una forma drástica.

¿Cuándo se producen sequías?

**A primera vista en el modelo descriptivo podemos analizar dos sequías drásticas, las mencionadas anteriormente, en los años, aproximadamente, 2002 y 2006. No obstante podemos analizarlo en profundidad.

**Crearemos un modelo que analize las sequías a partir del porcentaje del promedio de cada embalse

```
# PREGUNTA 2: Detectar sequías por embalse

import pandas as pd

# Cargar dataset pivotado

ruta_dataset = "/content/dive/MyDrive/dataset_embalses_final.csv"

df_pivot = pd.read_csv(ruta_dataset, sep=";", encoding="utf-8-sig")

# Columnas de embalses

columnas_embalses = df_pivot.columns.difference(['año', 'mes', 'TOTAL'])
```

```

# Convertir valores a float

df_embalses_float = df_pivot[columnas_embalses].replace(',', ' ', regex=True).astype(float)

# Definir umbral de sequía: 30% del promedio de cada embalse

umbrales = df_embalses_float.mean() * 0.3

# DataFrame booleano: True donde hay sequía para luego mostrar las fechas y el mes de las filas con valor True

sequias = df_embalses_float.lt(umbrales)

# Crear DataFrame con años, meses y sequías

df_sequia = df_pivot[['año', 'mes']].copy()

for embalse in columnas_embalses:

    df_sequia[embalse + '_sequía'] = sequias[embalse]

# Mostrar fechas de sequía para cada embalse

for embalse in columnas_embalses:

    fechas = df_sequia[df_sequia[embalse + '_sequía']][['año', 'mes']]

    print(f"\nEmbalse: {embalse} → meses con sequía:")

    print(fechas.to_string(index=False))

```

**Si modificamos el parámetro del porcentaje a 0.1 encontraremos los embalses con sequías extremas o críticas, sus meses y sus años.

Embalse: PuentesViejas → meses con sequía:

año	mes
2006	Enero
2006	Febrero
2009	Agosto
2009	Septiembre
2009	Octubre

Embalse: Riosequillo → meses con sequía:

año	mes
2006	Agosto
2006	Septiembre
2006	Octubre
2006	Noviembre
2007	Noviembre

Embalse: ElVillar → meses con sequía:

año	mes
2005	Mayo
2006	Marzo
2006	Abril
2006	Mayo
2006	Junio
2006	Julio
2007	Junio

**Estos embalses obtuvieron valores en los años y meses mostrados de tansolo el 10% del promedio de los embalses, es decir, si el promedio del embalse Villar es 100, su valor de sequía se detectará a partir del valor 10, y por debajo de este.

**Si ampliamos el valor del porcentaje podemos encontrar una sequía nueva, aparte de las mencionadas anteriormente

```
Embalse: LosMorales → meses con sequía:
año      mes
2009     Diciembre
2012     Septiembre
2012      Octubre
2012     Noviembre
```

**Al aumentar el valor al 30% (0.3) se obtienen demasiados resultados, los cuales no se definen como sequías sino valores más bajos de agua en embalses, es decir, el valor no es lo suficientemente bajo como para nombrarlo sequía.

**Repuesta final: Los embalses con sequías drásticas fueron PuentesViejas, en los años 2006 y 2009 y el embalse de Riosequillo, en los años 2006 y 2007 (en tan solo un mes).

**Podemos analizar los meses en concreto en las siguientes evidencias

**Sequías drásticas o críticas

```
Embalse: PuentesViejas → meses con sequía:
año      mes
2006      Enero
2006     Febrero
2009      Agosto
2009     Septiembre
2009      Octubre

Embalse: Riosequillo → meses con sequía:
año      mes
2006      Agosto
2006     Septiembre
2006      Octubre
2006     Noviembre
2007     Noviembre

Embalse: ElVillar → meses con sequía:
año      mes
2005      Mayo
2006      Marzo
2006      Abril
2006      Mayo
2006      Junio
2006      Julio
2007      Junio
```

**Sequías normales

```
Embalse: LosMorales → meses con sequía:
año      mes
2009     Diciembre
2012     Septiembre
2012      Octubre
2012     Noviembre
```

**Si analizamos las sequías que hubieron por año y no por embalse podemos analizar la gráfica de previsiones, los valores con mayor sequía fueron en 2002 y 2006

**Como conclusión se han detectado dos sequías extremas, en los años 2002 y 2006, también se han detectado sequías por embalse, las mencionadas anteriormente.

¿Qué embalse tiene más diferencia entre su máximo valor y su mínimo valor?

**Podemos crear nuevo código para obtener el embalse

```
# PREGUNTA 1: Embalse con mayor diferencia entre máximo y mínimo

import pandas as pd # Importaciones necesarias

# Cargar dataset pivotado (embalses en columnas)
ruta_dataset = "/content/dive/MyDrive/dataset_embalses_final.csv"
df_pivot = pd.read_csv(ruta_dataset, sep=";", encoding="utf-8-sig")

# Columnas de embalses
columnas_embalses = df_pivot.columns.difference(['año', 'mes', 'TOTAL'])

# Convertir valores a float
df_embalses_float = df_pivot[columnas_embalses].replace(',', '.', regex=True).astype(float)

# Calcular diferencia del máximo al mínimo por cada embalse
rango_embalses = df_embalses_float.max() - df_embalses_float.min()
```

```
# Embalse con mayor diferencia
embalse_max_diferencia = rango_embalses.idxmax()
valor_max_diferencia = rango_embalses.max()

print(f"El embalse con mayor diferencia entre máximo y mínimo es '{embalse_max_diferencia}' con una diferencia de {valor_max_diferencia:.2f} hm³.")
```

****El resultado es el siguiente**

```
El embalse con mayor diferencia entre máximo y mínimo es 'ElAtazar' con una diferencia de 275.66 hm³.
```

****El programa está obteniendo la diferencia del valor máximo de cada embalse y el valor mínimo y, da como resultado el embalse cual, su diferencia es mayor**

****Repuesta: El embalse con una mayor diferencia entre su valor máximo de agua y su valor mínimo es el embalse ElAtazar, con una diferencia de 275.66 hm³.**

3.- ¿Podemos asegurar una respuesta segura a la pregunta de la sequía con los datos que tenemos?

****Mediante los datos que tenemos podemos observar que los años de sequía extrema sí que son datos seguros, ya que los valores de todos los embalses en los años 2002 y 2006 caen drásticamente.**

****Respecto a las sequías por embalse podemos asegurar que las sequías extremas fueron en los siguientes años y meses de cada embalse.**

```
Embalse: PuentesViejas → meses con sequía:
año    mes
2006    Enero
2006    Febrero
2009    Agosto
2009    Septiembre
2009    Octubre

Embalse: Riosequillo → meses con sequía:
año    mes
2006    Agosto
2006    Septiembre
2006    Octubre
2006    Noviembre
2007    Noviembre

Embalse: ElVillar → meses con sequía:
año    mes
2005    Mayo
2006    Marzo
2006    Abril
2006    Mayo
2006    Junio
2006    Julio
2007    Junio
```

****A continuación se muestran estadísticas con las que podemos obtener una visión más amplia de los datos de cada embalse**

Evolución temporal de cada embalse

```
# ESTADÍSTICA 1: Evolución temporal de cada embalse

# Función de limpieza de valores bajos o nulos

def limpiar_valores_extremos(df_embalses):

    """Convierte a float y elimina valores cercanos a 0."""

    df_float = df_embalses.replace(',', '.', regex=True).astype(float)

    # Reemplazar valores menores o iguales a 0.5 hm³ por NaN

    df_float[df_float <= 0.5] = float('nan')

    return df_float
```

```

import pandas as pd

import matplotlib.pyplot as plt

ruta_dataset = "/content/dive/MyDrive/dataset_embalses_final.csv"

df_pivot = pd.read_csv(ruta_dataset, sep=";", encoding="utf-8-sig")

columnas_embalses = df_pivot.columns.difference(['anio', 'mes', 'TOTAL'])

# Limpiar datos

def limpiar_valores_extremos(df_embalses):

    df_float = df_embalses.replace(',', '.', regex=True).astype(float)

    df_float[df_float <= 0.5] = float('nan')

    return df_float

df_embalses_float = limpiar_valores_extremos(df_pivot[columnas_embalses])

# Crear fecha

meses_dict = {

    "Enero":1, "Febrero":2, "Marzo":3, "Abril":4, "Mayo":5, "Junio":6,

    "Julio":7, "Agosto":8, "Septiembre":9, "Octubre":10, "Noviembre":11, "Diciembre":12

}

df_pivot['mes_num'] = df_pivot['mes'].map(meses_dict)

df_pivot['fecha'] = pd.to_datetime(df_pivot['anio'].astype(str) + '-' + df_pivot['mes_num'].astype(str) +
'-01')

# Plot

plt.figure(figsize=(14,6))

for embalse in columnas_embalses:

    plt.plot(df_pivot['fecha'], df_embalses_float[embalse], label=embalse)

plt.title("Evolución temporal del volumen de agua (limpieza aplicada)")

plt.xlabel("Fecha")

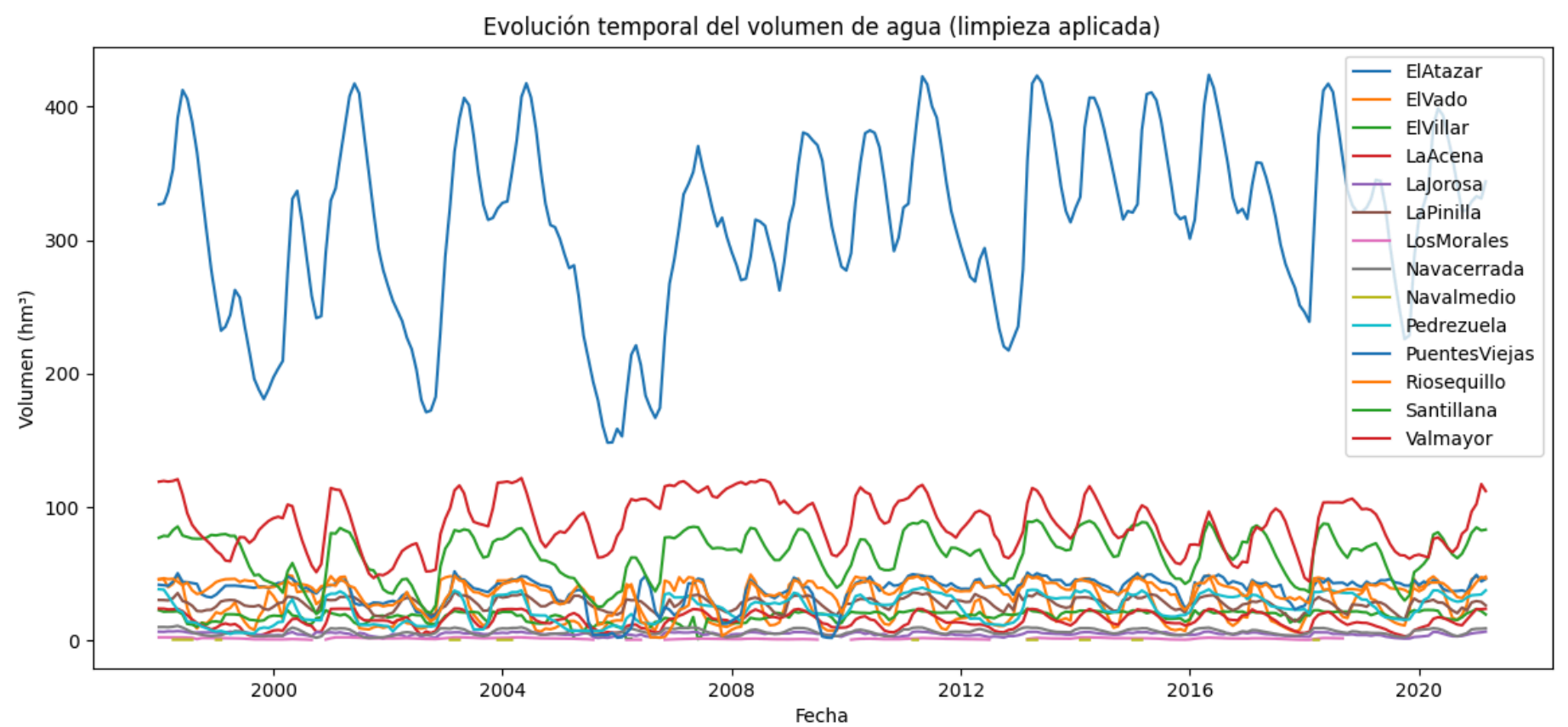
plt.ylabel("Volumen (hm³)")

plt.legend()

plt.show()

```

****Resultado**



**Cada lineal muestra la evolución de cada embalse según los años y la cantidad de agua, la leyenda de la derecha muestra el color de la lineal perteneciente a cada embalse.

Boxplot de volúmenes por embalse

```
# ESTADÍSTICA 2: Boxplot de volúmenes por embalse

import pandas as pd

import matplotlib.pyplot as plt

# Cargar dataset pivotado

ruta_dataset = "/content/dive/MyDrive/dataset_embalses_final.csv"

df_pivot = pd.read_csv(ruta_dataset, sep=";", encoding="utf-8-sig")

# Columnas de embalses

columnas_embalses = df_pivot.columns.difference(['año', 'mes', 'TOTAL'])

# Función para limpiar valores bajos o nulos

def limpiar_valores_extremos(df_embalses, umbral=0.5):

    """

    Convierte a float y reemplaza por NaN todos los valores <= umbral.

    Esto asegura que no se consideren para estadísticas o gráficos.

    """

    df_float = df_embalses.replace(',', '.', regex=True).astype(float)

    df_float[df_float <= umbral] = float('nan')

    return df_float

# Aplicar limpieza

df_embalses_float = limpiar_valores_extremos(df_pivot[columnas_embalses])
```

```
# Boxplot

plt.figure(figsize=(12,6))

df_embalses_float.boxplot()

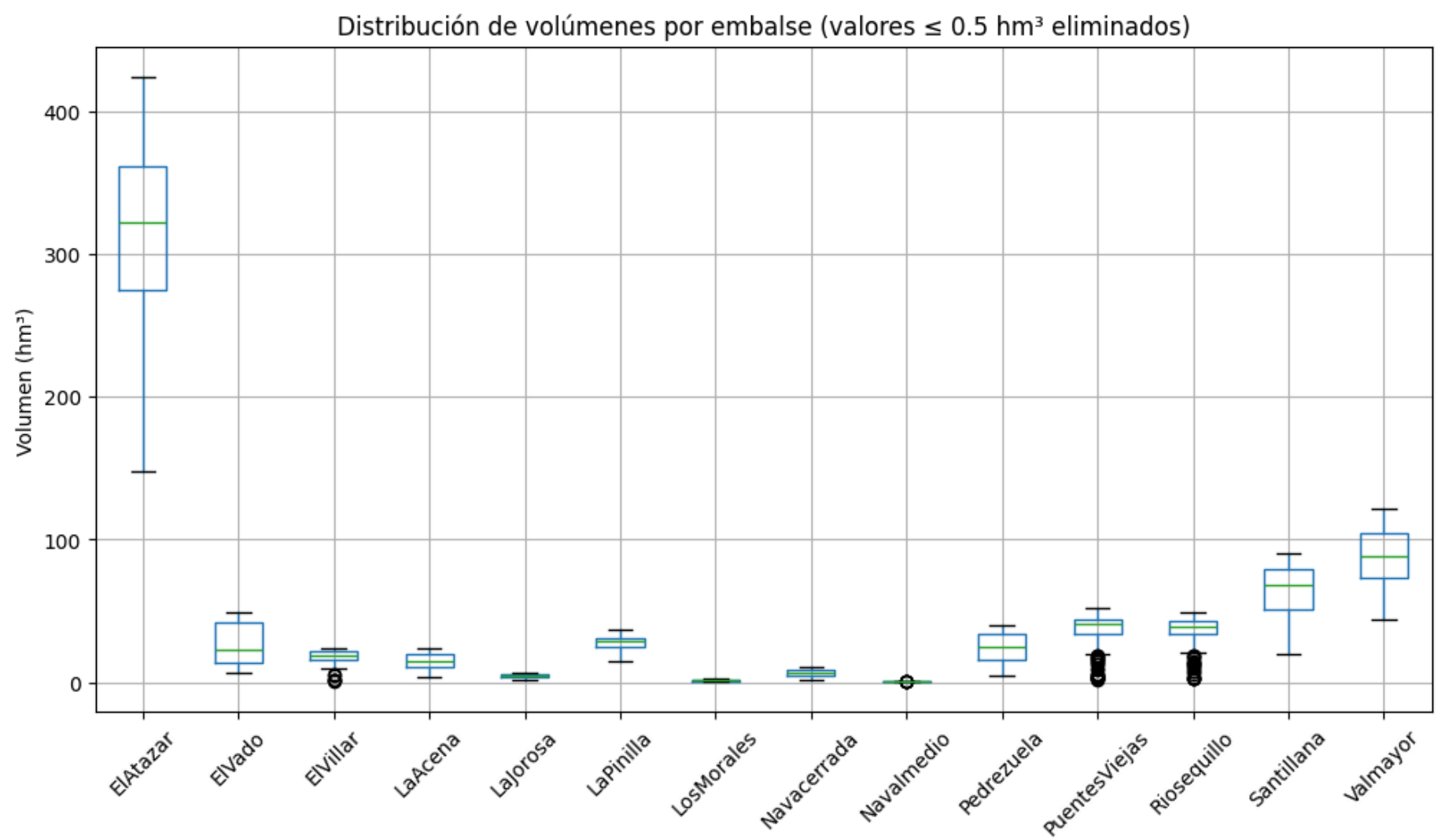
plt.title("Distribución de volúmenes por embalse (valores  $\leq 0.5$  hm3 eliminados)")

plt.ylabel("Volumen (hm3)")

plt.xticks(rotation=45)

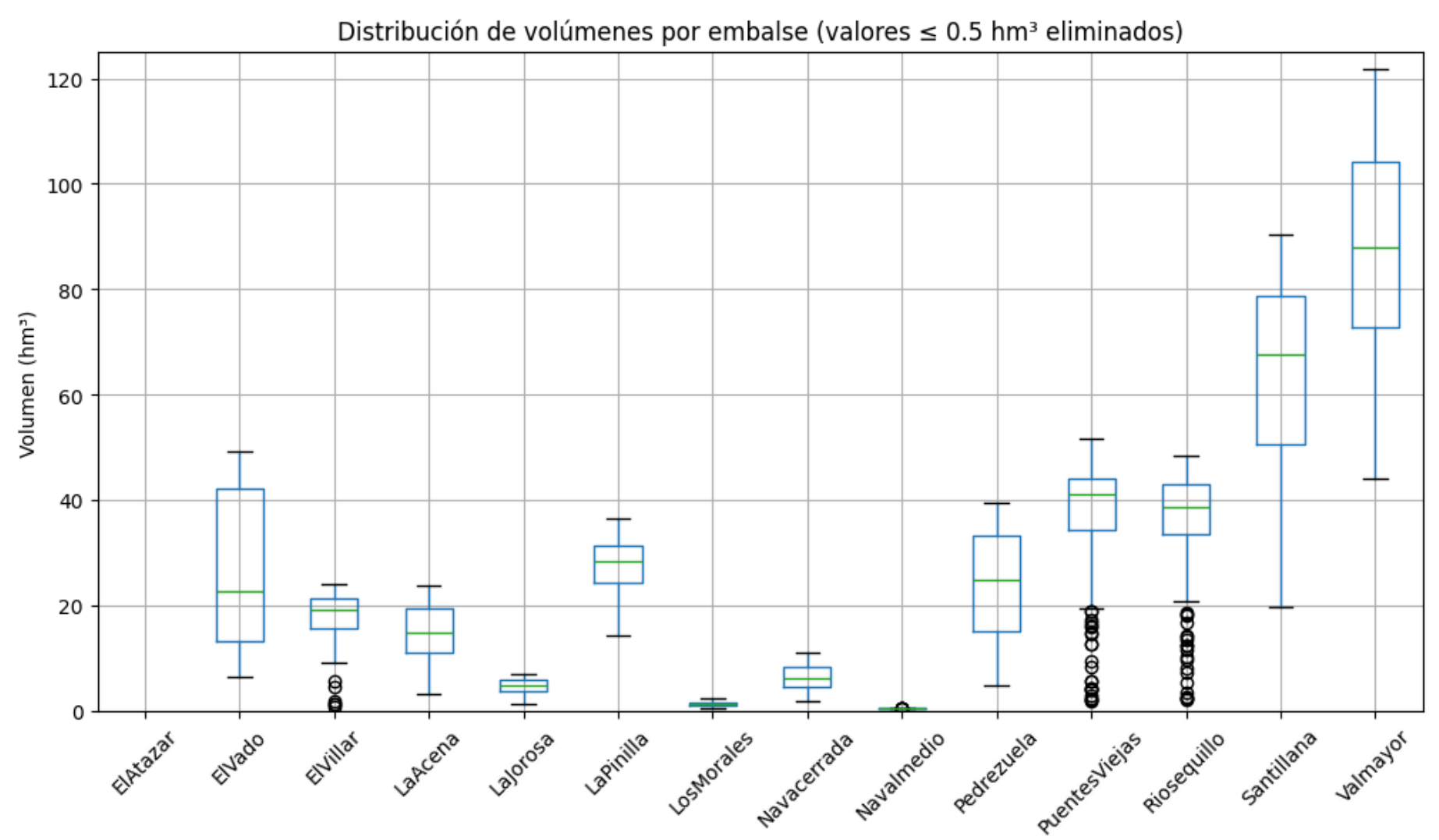
plt.show()
```

**Esta estadística crea una caja en la que se muestran los valores medios de un embalse, y a medida de que los vértices de la caja se alejan significa que los valores son muy superiores o muy inferiores. Puntos circulares indican anomalías en los datos, es decir, datos extremadamente bajos.

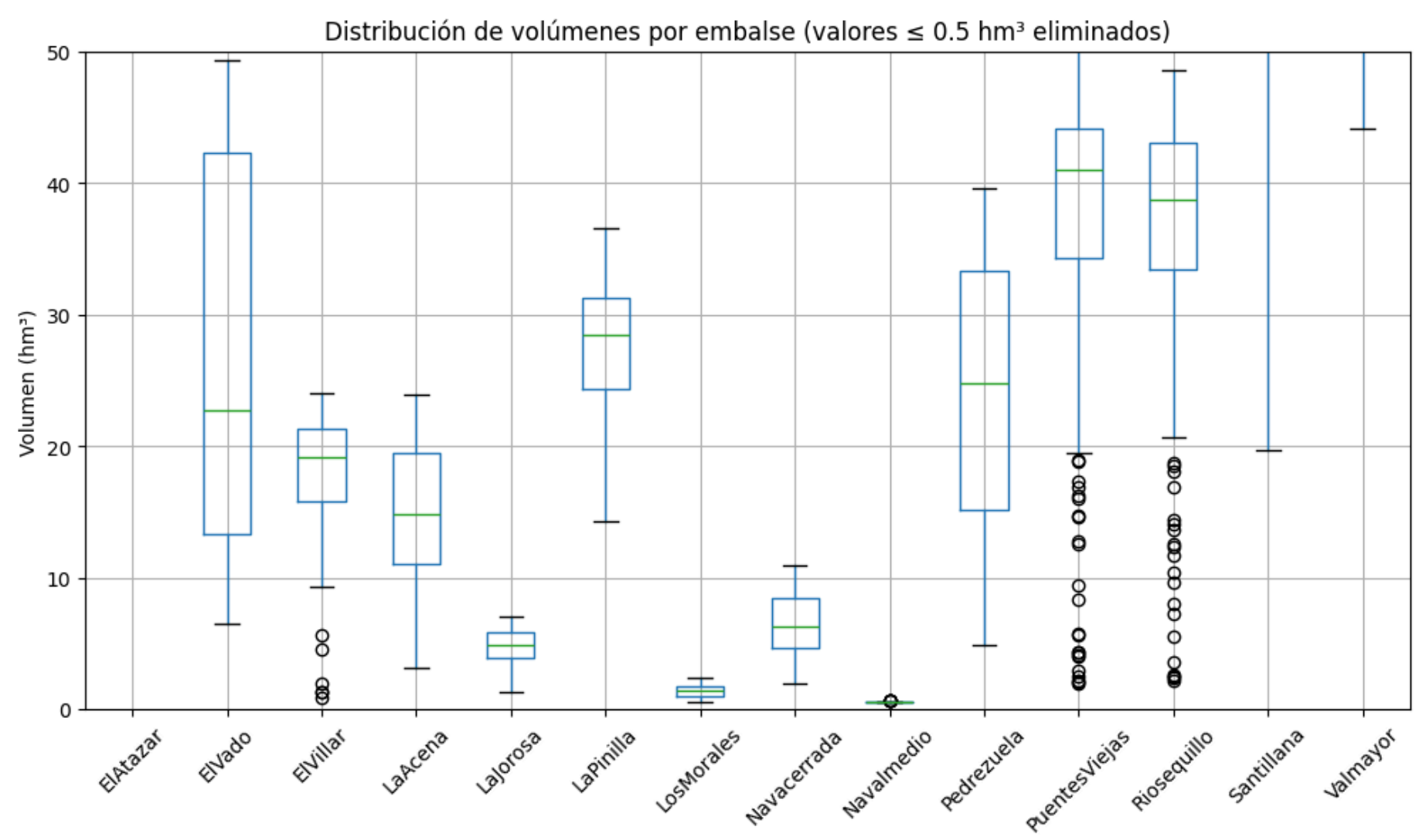


**La gráfica se ve compacta, podemos fijar los valores hasta 125 para eliminar ElAtazar

**Algunos valores fuera de las cajas son demasiado amplios, sobre todo los bajos, en Santillana y Valmayor. Por otra parte, los embalses Riosequillo, PuentesViejas, NavalMedio y Elvillar contienen datos extremadamente bajos (anomalías)



**Al decompactarla más podemos observar que algunos de los embalses contienen la mayoría de sus datos en 0 o rozando 0.



Histograma de volúmenes totales

```
# ESTADÍSTICA 3: Histograma de volúmenes totales

# Histograma con función específica de limpieza de valores inferiores

import pandas as pd

import matplotlib.pyplot as plt

ruta_dataset = "/content/dive/MyDrive/dataset_embalses_final.csv"

df_pivot = pd.read_csv(ruta_dataset, sep=";", encoding="utf-8-sig")
```

```

# Convertir TOTAL a float

df_pivot['TOTAL_float'] = df_pivot['TOTAL'].str.replace(',', '.').astype(float)

# Limpiar valores menores que definamos

df_pivot['TOTAL_float'] = df_pivot['TOTAL_float'].apply(lambda x: x if x >= 250 else float('nan'))

# Histograma

plt.figure(figsize=(10,5))

plt.hist(df_pivot['TOTAL_float'].dropna(), bins=20, color='skyblue', edgecolor='black')

plt.title("Distribución del volumen total")

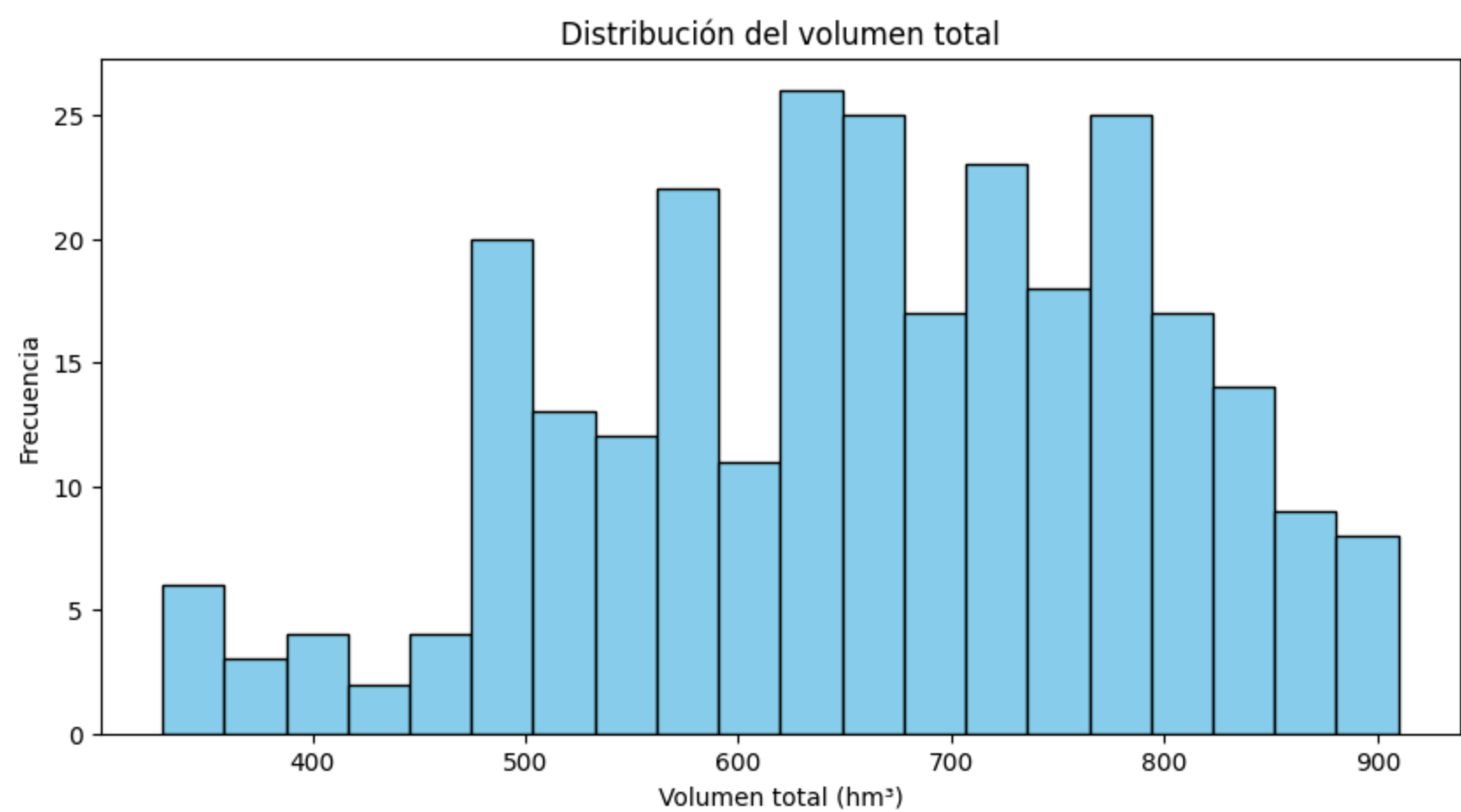
plt.xlabel("Volumen total (hm³)")

plt.ylabel("Frecuencia")

plt.show()

```

****Se muestran los volúmenes totales de los embalses y su frecuencia**



Hetmap de volúmenes por embalse y mes

****Mapa de calor que muestrala cantidad por embalse y mes en formato color**

****Se ha fijado como máximo en 100 debido a que la mayoría de los embalses rondan este valor como media**

```

# ESTADÍSTICA 4: HETMAP O MAPA DE CALOR POR EMBALSE Y MES

import pandas as pd

import matplotlib.pyplot as plt

import seaborn as sns

# Cargar dataset

ruta_dataset = "/content/dive/MyDrive/dataset_embalses_final.csv"

df_pivot = pd.read_csv(ruta_dataset, sep=";", encoding="utf-8-sig")

columnas_embalses = df_pivot.columns.difference(['año', 'mes', 'TOTAL'])

```

```

# Limpiar valores bajos

def limpiar_valores_extremos(df_embalses, umbral_min=0.5, umbral_max=100):

    df_float = df_embalses.replace(',', '.', regex=True).astype(float)

    df_float[df_float <= umbral_min] = float('nan') # eliminar valores muy bajos

    df_float[df_float > umbral_max] = umbral_max      # limitar valores muy altos

    return df_float


df_embalses_float = limpiar_valores_extremos(df_pivot[columnas_embalses])


# Crear pivot tabla para heatmap

df_heatmap = df_embalses_float.copy()

df_heatmap['fecha'] = pd.to_datetime(

    df_pivot['año'].astype(str) + '-' +

    df_pivot['mes'].map({

        "Enero":1, "Febrero":2, "Marzo":3, "Abril":4, "Mayo":5, "Junio":6,

        "Julio":7, "Agosto":8, "Septiembre":9, "Octubre":10, "Noviembre":11, "Diciembre":12

    }).astype(str) + '-01',

    format="%Y-%m-%d"

)


# Convertir el índice a string YYYY-MM para eliminar hora

df_heatmap = df_heatmap.set_index(df_heatmap['fecha'].dt.strftime('%Y-%m'))


plt.figure(figsize=(14,8))

sns.heatmap(df_heatmap[columnas_embalses].T, cmap='YlGnBu', cbar_kws={'label': 'Volumen (hm³)'})

plt.title("Heatmap de volúmenes por embalse y fecha")

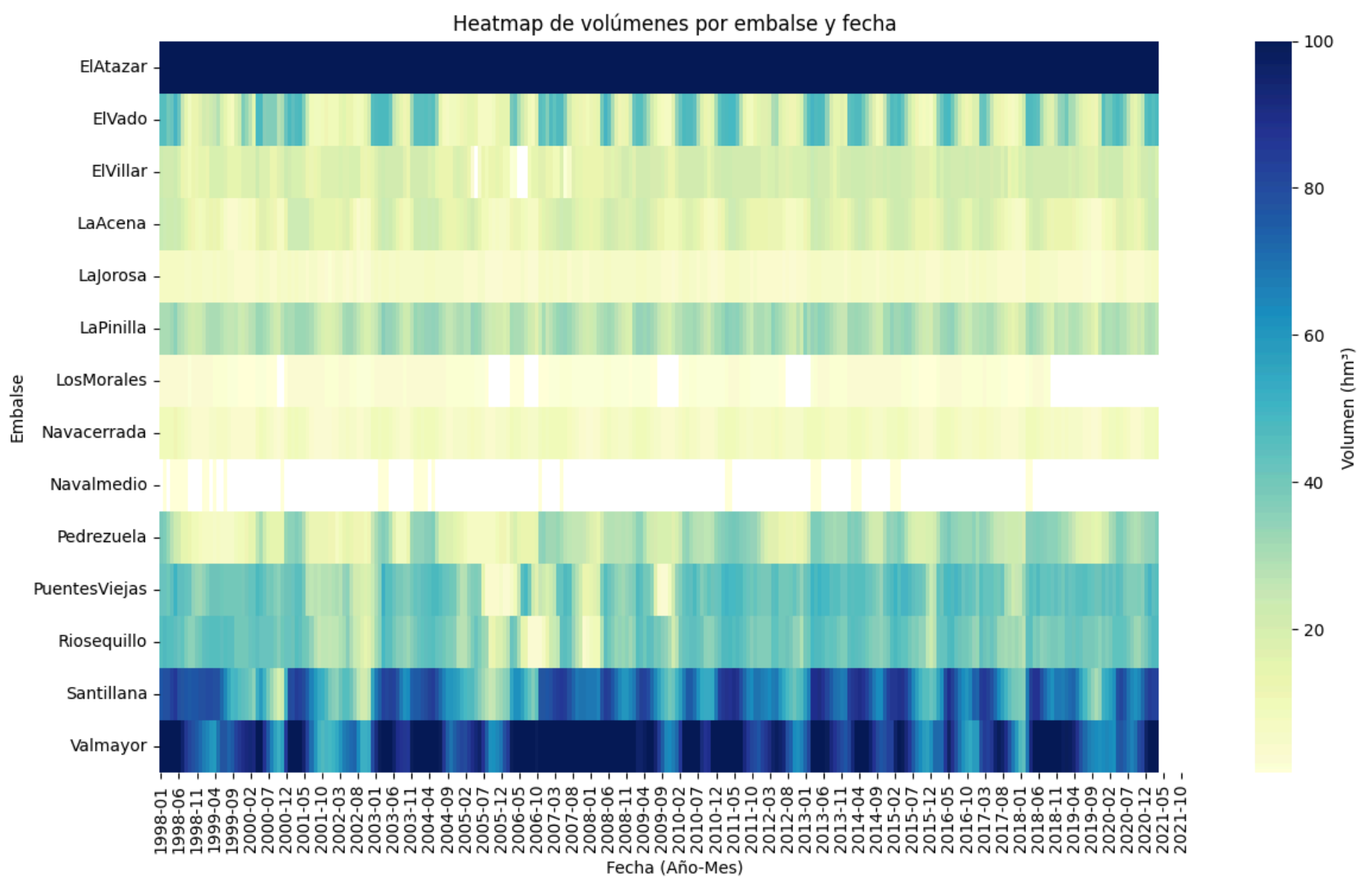
plt.xlabel("Fecha (Año-Mes)")

plt.ylabel("Embalse")

plt.show()

```

**Podemos ver que el orden de embalses por agua a simple vista es, de mayor a menor agua y siendo los primeros; ElAtzar, Valmayor y Santillana.



Linea de evolución del volúmen total

****Podemos ver como cambia el volúmen total en todos los embalses**

```
# Convertir TOTAL a float

df_pivot['TOTAL_float'] = df_pivot['TOTAL'].str.replace(',', ' ').astype(float)


# Limpiar valores menores a 0.5 hm³

df_pivot['TOTAL_float'][df_pivot['TOTAL_float'] <= 0.5] = float('nan')


# Crear fecha

df_pivot['fecha'] = pd.to_datetime(df_pivot['año'].astype(str) + '-' + df_pivot['mes'].map({
    "Enero":1, "Febrero":2, "Marzo":3, "Abril":4, "Mayo":5, "Junio":6,
    "Julio":7, "Agosto":8, "Septiembre":9, "Octubre":10, "Noviembre":11, "Diciembre":12
}).astype(str) + '-01')


# Plot evolución total

plt.figure(figsize=(14,6))

plt.plot(df_pivot['fecha'], df_pivot['TOTAL_float'], color='purple', marker='o')

plt.title("Evolución del volumen total de agua en todos los embalses")

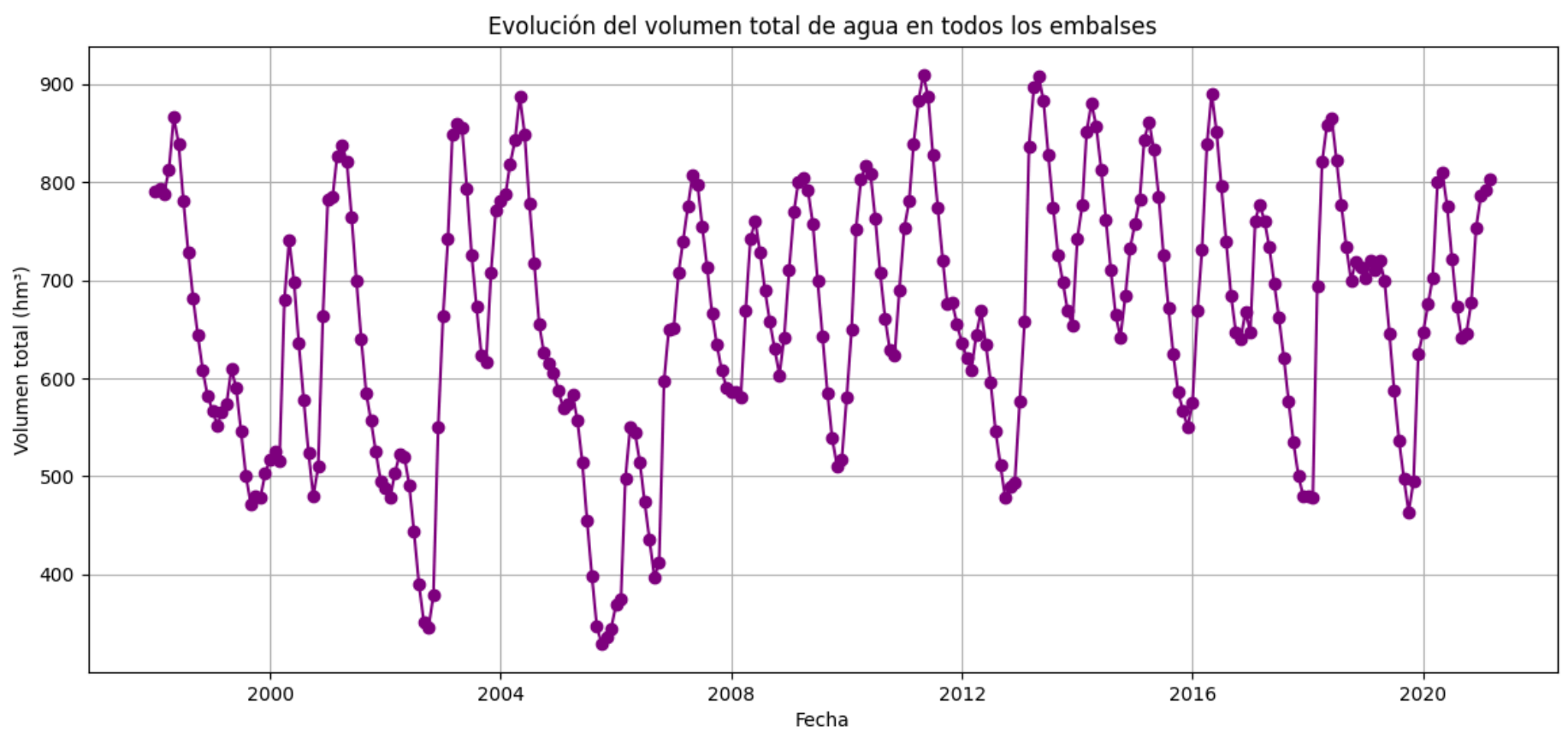
plt.xlabel("Fecha")

plt.ylabel("Volumen total (hm³)")

plt.grid(True)

plt.show()
```

**Se observan las sequías del año 2002 y el año 2006 y se pueden observar que lo cambios, fuera de estos años de sequías no son extremadamente drásticos.



Óscar Ortega Rueda